

Avaliação Técnica de uma Arquitetura Web baseada em BaaS/Serverless para Intermediação de Serviços

Um estudo de caso da plataforma Hubservi

Pedro Conrado Fernandes Vieira
Richardy Gabriel Rodrigues da Costa
Orientador: Prof. Daniel Facciolo Pires

O QUE É ESTE DOCUMENTO

Conteúdo **slide a slide** para a defesa oral, extraído do artigo *Artigo_PedroConrado_RichardyRodrigues_ATUALIZADO*. Para cada slide:

NA TELA — os bullets curtos que devem ir no slide, prontos para colar.

FALA — o roteiro do que dizer, no tom de quem apresenta.

FONTE NO ARTIGO — seção e tabela de origem, para conferência e para responder à banca.

Formato: 22 slides · 18 minutos · 2 apresentadores · 5 slides reserva para o Q&A.

A montagem visual e a estilização ficam por sua conta — aqui está apenas o conteúdo.

Divisão e cronometragem

Meta de **18:00**, com folga dentro da faixa de 15–20 minutos. Um único *handoff*, após o slide 11. **Regra de ouro:** se estiver atrasando, corte detalhe dos slides 7, 8 e 12; **nunca corte os slides 6, 18 e 21** — são o escopo, o achado central e a resposta à questão de pesquisa.

#	SLIDE	QUEM	TEMPO	ACUM.
1	Capa	Pedro	0:20	0:20
2	Agenda	Pedro	0:25	0:45
3	Contexto: intermediação de serviços e a ascensão do BaaS/Serverless	Pedro	0:50	1:35
4	O problema: a qualidade migra para configuração declarativa	Pedro	1:00	2:35
5	Questão de pesquisa e objetivos	Pedro	0:50	3:25
6	Objeto × instrumento — por que construímos o sistema	Pedro	1:05	4:30
7	Referencial I: arquitetura, atributos de qualidade e ATAM	Pedro	0:55	5:25
8	Referencial II: ISO/IEC 25010 e o paradigma BaaS	Pedro	0:55	6:20
9	Metodologia: classificação e as seis etapas	Pedro	0:55	7:15
10	Ferramentas e o ciclo medir → detectar → corrigir → re-medir	Pedro	0:55	8:10
11	Arquitetura do Hubservi: SPA + BaaS + Serverless	Pedro	0:55	9:05
handoff — Pedro → Richardy				
12	Modelo de dados e regras de negócio no banco	Richardy	0:50	9:55
13	Autorização declarativa via RLS	Richardy	0:50	10:45
14	Planejamento experimental: atributo → cenário → métrica → critério	Richardy	0:55	11:40
15	ATAM: árvore de utilidade e cenários priorizados	Richardy	0:55	12:35
16	ATAM: sensibilidade, compromissos e riscos	Richardy	0:50	13:25
17	Achado 1 — reprodutibilidade: as migrations não reconstroem o sistema	Richardy	0:50	14:15
18	Achado 2 — segurança: dois defeitos reais	Richardy	1:20	15:35
19	Confiabilidade e testabilidade	Richardy	0:55	16:30
20	Desempenho e manutenibilidade	Richardy	1:05	17:35
21	Síntese por atributo e resposta à questão de pesquisa	Richardy	0:55	18:30
22	Conclusão, limitações e trabalhos futuros · Perguntas	ambos	0:50	19:20

O total fecha em **19:20** — dentro da faixa de 15–20 minutos, mas perto do teto. Duas formas de ganhar folga, ambas sem perder nenhum resultado: **(a)** fundir os slides **7+8** em um só de referencial e os **12+13** em um só de arquitetura interna, economizando cerca de **1:45** e fechando em **17:35**; **(b)** no slide 22, deixar limitações e trabalhos futuros só na tela e falar apenas a contribuição, economizando **0:20**. Se a banca for rígida em 15 minutos, aplique as duas.

SLIDE 1 Capa

Pedro · 0:20 · acum. 0:20

NA TELA

- **Avaliação Técnica de uma Arquitetura Web baseada em BaaS/Serverless para Intermediação de Serviços**
- Um estudo de caso da plataforma Hubservi
- Pedro Conrado Fernandes Vieira · Richardy Gabriel Rodrigues da Costa
- Orientador: Prof. Daniel Facciolo Pires — Uni-FACEF · 2026

VISUAL — só o título, os nomes e a marca da instituição. Sem imagem de fundo concorrendo com o texto.

FALA

"Bom dia. Somos Pedro e Richardy, e nosso trabalho avalia tecnicamente uma arquitetura web baseada em BaaS e Serverless. Nos próximos minutos vamos mostrar *como* se avalia esse tipo de arquitetura — e o que encontramos quando aplicamos o método a um sistema real."

FONTE NO ARTIGO · folha de rosto

SLIDE 2 Agenda

Pedro · 0:25 · acum. 0:45

NA TELA

- 1 · O problema e a questão de pesquisa
- 2 · Referencial: ISO/IEC 25010 e ATAM
- 3 · Metodologia e o objeto avaliado
- 4 · A arquitetura do Hubservi
- 5 · O procedimento de avaliação
- 6 · **Resultados** — o que a avaliação encontrou
- 7 · Contribuição e conclusão

VISUAL — numeração vertical simples. Destaque visual no item 6, que é o núcleo da defesa.

FALA

"O caminho é este: começamos pelo problema e pela questão de pesquisa, passamos rapidamente pelo referencial e pela metodologia, mostramos a arquitetura avaliada e o procedimento que construímos — e o maior tempo fica nos resultados, que é onde o trabalho se prova."

FONTE NO ARTIGO · §1.7 Organização do artigo

NA TELA

- Intermediação de serviços migrou para plataformas digitais — dados pessoais e transações entre desconhecidos
- **Serverless**: provisão, escala e manutenção de servidores delegadas ao provedor
- **BaaS**: autenticação, banco, storage e autorização como serviços gerenciados, consumidos direto pelo cliente
- No Supabase: PostgreSQL + **Row Level Security** + views + triggers
- Resultado: some a camada de servidor de aplicação própria

VISUAL — dois blocos lado a lado: à esquerda, arquitetura tradicional (Cliente → Servidor de aplicação → Banco); à direita, BaaS (Cliente → BaaS/Banco com RLS). A seta que some é o argumento inteiro.

FALA

"O modelo de desenvolvimento web foi reconfigurado pela popularização do BaaS e do serverless. Responsabilidades que antes ficavam em um servidor próprio — autenticação, persistência, autorização — passam a ser serviços gerenciados, acessados diretamente pelo navegador."

"No Supabase, isso se concretiza sobre um PostgreSQL: a autorização vira *Row Level Security*, política declarativa avaliada pelo próprio banco a cada operação. Repare no que desaparece do desenho: o servidor de aplicação intermediário. E é exatamente esse desaparecimento que cria o problema do nosso trabalho."

FONTE NO ARTIGO · §1.1 Contexto; §2.4 Arquiteturas BaaS e Serverless

NA TELA

- Sem servidor próprio, **segurança e confiabilidade** passam a viver em:
 - políticas de RLS
 - triggers e restrições de integridade
 - a fronteira cliente ↔ serviço gerenciado
- Muda **o que** testar, **onde** estão os pontos de falha e **como** medir
- Falha de RLS não gera erro funcional aparente — a tela continua funcionando
- **Lacuna**: ATAM e ISO/IEC 25010 foram concebidos supondo servidor de aplicação explícito

FALA

"O problema é que essa simplificação tem um custo escondido. Atributos de qualidade críticos — segurança e confiabilidade acima de tudo — deixam de ser código e viram *configuração declarativa*. E configuração errada não quebra a aplicação: a tela continua funcionando normalmente enquanto o dado vaza."

"Some-se a isso que os métodos clássicos de avaliação arquitetural — o ATAM, a ISO 25010 — foram pensados sob o pressuposto de que existe uma camada de servidor de aplicação. Existe, portanto, uma lacuna prática: não é evidente como aplicar métricas e testes de Engenharia de Software para validar esses atributos nesse novo arranjo."

Enunciado do problema, se quiser ler: "Equipes de desenvolvimento têm dificuldade em validar tecnicamente se uma arquitetura baseada em BaaS e Serverless atende aos atributos de qualidade esperados."

FONTE NO ARTIGO · §1.2 Problema de pesquisa; §1.6 Justificativa; §2.4

NA TELA

- **Questão:** Como avaliar tecnicamente uma arquitetura web baseada em BaaS/Serverless por meio de métricas e testes de Engenharia de Software, considerando atributos de qualidade?
- **Objetivo geral:** avaliar tecnicamente a arquitetura do Hubservi (React, TypeScript, Supabase, PostgreSQL)
- **Nove objetivos específicos**, em três frentes:
 - OE1–OE4 — identificar atributos, modelar, documentar, definir cenários
 - OE5–OE8 — executar testes, análise estática, segurança e desempenho
 - OE9 — analisar os resultados
- Todos os nove **executados** — reportados na Seção 7

FALA

"Daí decorre a questão de pesquisa, que é uma pergunta de *como*, não de *quanto*: como avaliar tecnicamente uma arquitetura BaaS/Serverless por meio de métricas e testes de Engenharia de Software."

"O objetivo geral é aplicar essa avaliação à plataforma Hubservi. Os nove objetivos específicos se organizam em três frentes — modelar e definir cenários; executar as medições; e analisar. Registro desde já que os nove foram executados, inclusive antecipando o cronograma original, e os resultados estão na Seção 7 do artigo."

FONTE NO ARTIGO · §1.3 Questão de pesquisa; §1.4 Objetivos; nota de escopo em §1.4.2

NA TELA

- **Objeto de pesquisa** = a **avaliação** da arquitetura → é a contribuição científica
- **Instrumento** = a plataforma Hubservi → é o meio, não o fim
- Por que desenvolver o sistema? Avaliar RLS, triggers e a fronteira cliente–banco exige acesso irrestrito a código, schema, configuração e ambiente — **impossível sobre plataforma de terceiros**
- É o que garante a **validade interna**: controlar o ambiente, reproduzir sobre estado conhecido, corrigir defeitos sem restrição
- **Fora do escopo**: usabilidade e acessibilidade · módulo de recomendação · microsserviços · E2E por automação de navegador

VISUAL — dois cartões contrastados: "OBJETO — a avaliação" em destaque forte, "INSTRUMENTO — o Hubservi" em tom secundário. A hierarquia visual precisa dizer sozinha qual dos dois é a pesquisa.

FALA

"Este slide é o que evita o mal-entendido mais comum sobre o nosso trabalho. Existem dois produtos aqui, de naturezas diferentes: um sistema de software, que desenvolvemos integralmente, e um procedimento de avaliação, aplicado sobre esse sistema. **Só o segundo é a contribuição científica.**"

"Então por que construir o sistema? Porque avaliar políticas de RLS, triggers e a fronteira entre cliente e serviço gerenciado exige acesso irrestrito ao código-fonte, ao esquema do banco e ao ambiente de execução. Isso é inviável sobre uma plataforma de terceiros. O desenvolvimento próprio não é o objetivo do trabalho — é pré-requisito metodológico dele, e é o que assegura a validade interna do experimento."

"E delimitamos negativamente: usabilidade e acessibilidade ficam fora, porque estão fora das quatro características da 25010 que selecionamos; recomendação ficou como módulo secundário; e não avaliamos microsserviços, porque o sistema simplesmente não os tem."

FONTE NO ARTIGO · §1.5 Delimitação do escopo (objeto e instrumento; justificativa do desenvolvimento; delimitação negativa)

NA TELA

- **Bass, Clements e Kazman (2012)** — a arquitetura é o artefato que viabiliza ou inibe os atributos de qualidade
- Cenário de atributo de qualidade em **seis partes**: fonte · estímulo · artefato · ambiente · resposta · medida
- **ATAM** (Clements, Kazman e Klein, 2002) — avaliação por cenários; não dá nota, identifica:
 - **ponto de sensibilidade** — propriedade crítica para um atributo
 - **ponto de compromisso** — sensível a mais de um atributo; melhorar um degrada outro
 - **riscos e não riscos**
- Organizado por uma **árvore de utilidade** com cenários priorizados

FALA

"Dois referenciais sustentam o método. O primeiro é Bass, Clements e Kazman: a arquitetura é o artefato em que os atributos de qualidade se tornam analisáveis, e esses atributos se expressam por cenários estruturados em seis partes. Essa estrutura é o que nos permitiu transformar 'o sistema é seguro' em algo verificável."

"O segundo é o ATAM. O ponto importante do ATAM é que ele *não* atribui notas: ele identifica riscos, pontos de sensibilidade e pontos de compromisso. Ele é especialmente adequado aqui porque arquiteturas BaaS são feitas de compromissos explícitos — você troca controle por simplicidade, e alguém precisa dizer o que se perdeu na troca."

FONTE NO ARTIGO · §2.1 Arquitetura e atributos de qualidade; §2.2 Avaliação arquitetural e ATAM

NA TELA

- **ISO/IEC 25010 (2011)**, família SQuaRE — 8 características de qualidade de produto
- Quatro selecionadas para este trabalho:
 - **Segurança** — confidencialidade, integridade, autenticidade
 - **Eficiência de desempenho** — comportamento temporal, capacidade
 - **Manutenibilidade** — modularidade, analisabilidade, **testabilidade**
 - **Confiabilidade** — maturidade, tolerância a falhas
- Testabilidade é subcaracterística da manutenibilidade — **destacada** por ser condição para verificar as demais
- Implicações do BaaS: autorização vira superfície declarativa · desempenho percebido vai para o cliente · testar exige alcançar o banco

FALA

"A ISO 25010 fornece a taxonomia. Das oito características do modelo, selecionamos quatro como relevantes ao problema: segurança, eficiência de desempenho, manutenibilidade e confiabilidade. Cada métrica que coletamos está rastreada a uma delas — não há métrica solta no trabalho."

"Uma observação: na norma, testabilidade é subcaracterística de manutenibilidade. Nós a tratamos como dimensão destacada, porque ela é condição de possibilidade para verificar todos os outros atributos. Se você não consegue testar, não consegue avaliar nada."

"E o paradigma BaaS traz três implicações diretas para a avaliação: a autorização vira uma superfície declarativa que precisa ser testada explicitamente; parte do desempenho percebido se desloca para o carregamento do cliente; e a testabilidade passa a depender de conseguir testar as regras que moram dentro do banco."

FONTE NO ARTIGO · §2.3 ISO/IEC 25010; §2.4 implicações arquiteturais; §2.5 testes e análise estática

NA TELA

- **Natureza** aplicada · **Abordagem** mista · **Objetivos** exploratórios e descritivos · **Procedimento** estudo de caso com experimento técnico
- Seis etapas:
 - 1 · Levantar requisitos arquiteturais e atributos de qualidade (ASR)
 - 2 · Modelar a arquitetura real — a partir do código e das migrations
 - 3 · Documentar componentes, fluxos, persistência e regras
 - 4 · Definir cenários — árvore de utilidade ATAM + plano de métricas
 - 5 · **Executar** — testes, análise estática, segurança, desempenho
 - 6 · Analisar em face dos critérios e dos tradeoffs
- Ambiente: instância **Supabase local em contêiner** — reprodutível e isolado de produção

FALA

"A pesquisa é aplicada, de abordagem mista, exploratória e descritiva, conduzida como estudo de caso combinado a experimento técnico. Mista porque combina métricas objetivas — cobertura, latência, complexidade — com a análise qualitativa das decisões arquiteturais via ATAM."

"A execução se organiza em seis etapas. Chamo atenção para a etapa 2: a arquitetura não foi *projetada* no papel e depois avaliada; ela foi **recuperada do código-fonte e das migrations**, para que a modelagem representasse o sistema real e não uma intenção. E toda a medição de banco e de API ocorreu contra uma instância Supabase local em contêiner — nunca contra dados de produção."

FONTE NO ARTIGO · §3.1 Classificação (Tabela 1); §3.3 Etapas metodológicas; §5.4 Ambiente

NA TELA

- **Testes** Vitest + React Testing Library · integração contra a API PostgREST
- **Análise estática** ESLint · eslint-plugin-sonarjs · jscpd · Madge · dependency-cruiser
- **Desempenho** Lighthouse (carregamento) · autocannon (carga)
- **Segurança** testes de RLS por papel · npm audit · supabase db lint · *OWASP ZAP pendente*
- Substituições registradas: SonarQube → sonarjs/jscpd · k6/JMeter → autocannon · Snyk → npm audit
- **Ciclo adotado:** medir → detectar → corrigir → **re-medir**, reportando o par antes/depois
- **Regra anti-fabricação:** nenhum valor sem evidência arquivada; resultado desfavorável registrado com o mesmo rigor

VISUAL — o ciclo de quatro caixas em círculo ocupando metade do slide; as ferramentas em lista compacta na outra metade.

FALA

"As ferramentas estão organizadas por atributo. Onde a ferramenta prevista no plano não estava disponível no ambiente — porque exigia conta ou serviço externo — trocamos por um substituto da mesma classe, medindo as mesmas métricas, e registramos cada troca. Nada foi silenciosamente abandonado."

"Mas o ponto metodológico mais importante deste slide é o ciclo. Adotamos **medir, detectar, corrigir e re-medir**, preservando os dois estados como evidência. A razão é direta: um conjunto de cenários que aprovasse o sistema já na primeira execução não demonstraria qualidade — demonstraria fraqueza dos cenários."

"E vale como declaração à banca: toda medição registra ferramenta, versão, ambiente, configuração, data, valor e veredito, com o artefato bruto arquivado. Nenhum número que vocês vão ver existe sem evidência associada."

FONTE NO ARTIGO · §3.4 Instrumentos; §3.5 Procedimentos de análise (regra anti-fabricação); §5.3 Tabela 12

NA TELA

- Estilo **SPA + BaaS + Serverless** — sem servidor de aplicação próprio, sem microsserviços
- Quatro camadas lógicas:
 - **Apresentação** — `src/pages/` , `src/components/`
 - **Aplicação** — AuthContext, React Query, schemas Zod
 - **Integração** — `src/integrations/supabase/`
 - **Dados** — `supabase/migrations/` (PostgreSQL)
- Stack: React 18.3 · TypeScript 5.8 · Vite 5.4 · React Query 5.83 · supabase-js 2.98
- Quatro fluxos críticos: **autenticação · serviços · booking · review**

VISUAL — usar a Figura 1 do artigo (camadas lógicas). É o slide mais visual do bloco A; deixe o diagrama grande e os bullets curtos.

FALA

"O Hubservi adota o estilo SPA + BaaS + Serverless. O SPA roda no navegador e conversa diretamente com o Supabase, que provê autenticação, o PostgreSQL e a autorização declarativa por RLS. Não há servidor de aplicação intermediário e não há microsserviços — e isso corrige, inclusive, uma incoerência documental anterior do próprio projeto, que mencionava microsserviços inexistentes."

"São quatro camadas lógicas, cada uma com elementos identificáveis no repositório, e quatro fluxos críticos: autenticação, cadastro e busca de serviços, contratação e avaliação. São esses quatro fluxos que a avaliação vai exercitar."

Handoff: "A partir daqui, o Richardy apresenta como avaliamos essa arquitetura e o que encontramos."

FONTE NO ARTIGO · §4.1 Visão geral (Figura 1, Tabela 2); §4.2 Tecnologias (Tabela 3); §4.5 Fluxos

HANDOFF — PEDRO → RICHARDY

Combine o gesto: Pedro devolve o controle do slide e senta. Uma frase só, sem "agora vou passar para o meu colega que vai falar sobre...".

SLIDE 12 Regras de negócio dentro do banco

Richardy · 0:50 · acum. 9:55

NA TELA

- Esquema: **5 tabelas** (profiles, categories, services, bookings, reviews) · **3 enums** · **2 views**
- Views: `service_stats` (contagem e média de avaliações) · `public_profiles` (perfil **sem PII**)
- Invariantes impostas por **trigger**, não pelo cliente:
 - provisão automática de perfil no cadastro (idempotente)
 - máquina de estados do booking
 - imutabilidade do `user_type` — anti-escalonamento
 - `booking.provider_id` = dono do serviço
- Máquina de estados: `pending` → `accepted` | `rejected` | `cancelled` · `accepted` → `completed` | `cancelled`

FALA

"Antes dos resultados, é preciso ver *onde* mora a regra de negócio nessa arquitetura. O esquema tem cinco tabelas, três enumerações e duas views. E boa parte das invariantes do domínio é imposta no servidor, por triggers e restrições — não no cliente. Essa é a marca do paradigma BaaS."

"Por exemplo: o tipo de usuário é imutável por trigger, o que impede escalonamento de privilégio; o prestador de um booking tem que coincidir com o dono do serviço; e a máquina de estados do booking rejeita qualquer transição fora do desenho. Guardem esses mecanismos — eles reaparecem nos resultados, tanto no que funcionou quanto no que faltava."

FONTE NO ARTIGO · §4.4 Modelo de dados; §4.6 Regras no banco (Tabela 5)

NA TELA

- **Todas** as tabelas com RLS habilitado — política avaliada pelo PostgreSQL a cada operação
- `profiles` — cada usuário lê e edita apenas o próprio registro; contraparte vem de `public_profiles`
- `services` — todos veem os ativos; só o prestador dono altera
- `bookings` — cliente e prestador veem os próprios; só o cliente insere; cliente cancela apenas os pendentes
- `reviews` — inserção só com booking `completed`; edição só pelo autor
- Toda a autorização concentrada em um único mecanismo declarativo

VISUAL — a Tabela 6 do artigo, reduzida a tabela/operação/política. Se ficar densa, mostre só profiles, bookings e reviews.

FALA

"Toda a autorização do sistema é declarativa. Não existe um `if` em JavaScript decidindo quem pode ler o quê: existe uma política SQL que o PostgreSQL avalia a cada operação."

"Isso é elegante e é consistente — qualquer caminho de acesso passa pela mesma regra. Mas concentra toda a correção da segurança em um único mecanismo. Uma política ausente ou mal escrita compromete o sistema inteiro sem produzir nenhum erro visível. É precisamente esse o ponto que a nossa avaliação de segurança foi construída para exercitar."

(Se quiser antecipar tensão para o slide 18:) "E adianto: a tabela que vocês estão vendo já está no estado **posterior** às correções."

FONTE NO ARTIGO · §4.7 Segurança: autorização declarativa via RLS (Tabela 6); §4.8 Síntese arquitetural

NA TELA

- Para cada atributo: **cenário** → **métrica** → **critério** → **ferramenta** → **coleta** → **veredito**
- Critérios **fixados antes da coleta** — são metas de planejamento, não resultados
- Cenários definidos: **7** de segurança · **4** de desempenho · **3** de testabilidade · **4** de manutenibilidade · **3** de confiabilidade
- Exemplos de critério: 0 acessos indevidos · $p95 \leq 800$ ms e erro $< 1\%$ · $LCP \leq 2,5$ s e score ≥ 90 · duplicação $\leq 3\%$ · 0 dependências circulares
- Cada execução registra ferramenta, versão, ambiente, configuração, data, valor e veredito

VISUAL — a Figura 2 do artigo (fluxo do procedimento) como faixa horizontal no topo, e abaixo dois ou três exemplos de linha das Tabelas 7 a 11.

FALA

"Cada atributo da 25010 foi decomposto em cenários mensuráveis. Para cada cenário definimos a métrica, o critério de aceitação e a ferramenta — e isso foi fixado **antes** de medir qualquer coisa. Esse detalhe importa: somos a própria equipe desenvolvedora avaliando o próprio sistema, e fixar critério antes da coleta é o que impede acomodar a meta ao resultado."

"Um único limiar ficou em aberto no planejamento, o de latência. Ele foi calibrado antes da coleta definitiva, a partir de uma execução exploratória de baixa intensidade e da heurística de um segundo para percepção de fluidez, adotando margem conservadora — chegamos a 800 milissegundos."

"Vale registrar também um refinamento honesto: o cenário de exposição de dados pessoais originalmente considerava só o usuário anônimo. Ao inspecionar as políticas vigentes, vimos que ele passaria sem exercitar o perfil de acesso mais permissivo que de fato existia. Ampliamos o cenário para incluir o usuário autenticado não relacionado — porque um cenário incapaz de reprovar o sistema não é instrumento de medição."

FONTE NO ARTIGO · §5.1 Visão geral (Figura 2); §5.2 Tabelas 7 a 11; nota de refinamento em §5.2.1

NA TELA

- Árvore decompõe: utilidade → atributos → refinamentos → cenários, rotulados por **(importância, risco)** em A/M/B
- Dez cenários priorizados:
 - **CS-1** isolamento de dados por RLS (**A, A**) · **CS-2** anti-escalamento (A, M) · **CS-3** não exposição de PII (A, M)
 - **CD-1** carregamento da SPA (M, M) · **CD-2** busca sob carga (A, M)
 - **CM-1** complexidade e duplicação (M, B) · **CM-2** sem ciclos (M, B)
 - **CC-1** máquina de estados (A, M) · **CC-2** integridade em cascata (A, B)
 - **CT-1** cobertura dos fluxos críticos (M, M)
- Formato de seis partes — ex. **CS-3**: fonte visitante/autenticado não relacionado · estímulo leitura de perfil · artefato `public_profiles` + RLS · resposta só campos não sensíveis · **medida 0 ocorrências de email ou phone**

VISUAL — a Figura 3 (árvore de utilidade). Se não couber legível, mostre só os três ramos de segurança e cite os demais.

FALA

"O ATAM entra para organizar a análise qualitativa. A árvore de utilidade decompõe a utilidade geral do sistema até chegar em cenários, cada um rotulado por importância para o negócio e risco arquitetural."

"Dez cenários foram priorizados. Só um recebeu **alto em ambos os eixos**: o CS-1, isolamento de dados por RLS — e vocês vão ver, nos resultados, que era exatamente ali que os defeitos estavam."

"Cada cenário é especificado em seis partes. No CS-3, por exemplo: a fonte é um visitante ou um usuário autenticado não relacionado; o estímulo é ler dados de perfil; o artefato é a view `public_profiles` e a RLS de profiles; e a medida é zero ocorrências de e-mail ou telefone expostos. É isso que torna 'ser seguro' uma afirmação falseável."

FONTE NO ARTIGO · §6.2 Árvore de utilidade (Figura 3); §6.3 Cenários em seis partes

NA TELA

- **Sensibilidade** — a segurança depende inteiramente da completude e exatidão das políticas de RLS e triggers; uma política ausente compromete CS-1 e CS-3 **sem erro funcional aparente**
- **Sensibilidade** — o desempenho de CD-2 depende da eficiência da view `service_stats` e da indexação
- **Compromisso** — delegar autorização ao banco favorece simplicidade e consistência, mas concentra o risco em um único mecanismo e desloca o teste para a fronteira cliente–banco
- **Compromisso** — o SPA acelera o desenvolvimento, porém transfere ao cliente o desempenho percebido
- **Risco** — carregamento inicial não atinge a meta (confirmado em §7.3.5)
- **Não risco** — imutabilidade de `user_type` e validação de `provider_id` dão defesa em profundidade

FALA

"Da análise qualitativa saíram dois pontos de sensibilidade, dois pontos de compromisso, um risco e um não risco. Destaco o primeiro ponto de sensibilidade, porque ele foi **confirmado empiricamente** pelas medições: a correção da segurança é altamente sensível à completude das políticas, e uma política mal especificada não gera erro visível."

"E os dois compromissos são a assinatura do paradigma: você ganha simplicidade delegando autorização ao banco, mas concentra o risco; ganha velocidade com o SPA, mas paga no desempenho percebido pelo usuário. Nos próximos slides, as medições dizem quanto se pagou em cada um."

FONTE NO ARTIGO · §6.4 Pontos de sensibilidade, de compromisso e riscos (Tabela 13)

NA TELA

- Ao reconstruir o banco **só a partir das migrations versionadas**, o sistema não funcionava
- Faltavam os **grants** de privilégio de API aos papéis `anon`, `authenticated` e `service_role` no schema `public`
- Resultado: `permission denied` **antes mesmo** de a RLS ser avaliada
- Funciona em produção porque esses privilégios foram aplicados **fora do controle de versão**
- Corrigido por migration que torna o histórico auto-suficiente — sem alterar políticas de segurança
- **Achado característico do BaaS: parte da configuração vive no serviço gerenciado e escapa ao versionamento**

FALA

"O primeiro achado não estava previsto em nenhum cenário — ele apareceu porque insistimos em montar um ambiente limpo e reproduzível."

"Ao reconstruir o banco apenas a partir das migrations versionadas, o sistema não subia: faltavam os *grants* de privilégio de API aos papéis do Supabase. O erro era `permission denied`, e ele acontecia **antes** de qualquer política de RLS ser avaliada."

"O sistema funciona em produção porque alguém aplicou esses privilégios manualmente, pelo painel, fora do versionamento. Ou seja: o repositório não era capaz de reconstruir o sistema do zero, e ninguém sabia disso."

"Este é um achado **característico do paradigma**: em BaaS, parte da configuração vive no serviço gerenciado e escapa ao controle de versão. E ele só se revela sob uma avaliação conduzida em ambiente limpo — o que é, em si, um argumento a favor do método."

NA TELA

- **Defeito 1 — PII exposta a qualquer usuário autenticado**
política de leitura de `profiles` usava `USING (auth.uid() IS NOT NULL)` → qualquer logado lia **email e telefone de todos**
Correção: leitura direta restrita ao próprio registro; contraparte via `public_profiles`
- **Defeito 2 — avaliação atribuída a prestador incorreto**
a política de inserção de `reviews` não validava `provider_id` contra o dono do serviço — assimetria em relação a bookings, que já tinham o trigger
Correção: trigger de validação espelhado
- **Após correção e re-medição: 0 acessos indevidos** — 30 testes de integração em 9 arquivos
- Dependências (npm audit): 12 alta · 8 moderada · **0 crítica**; apenas **1 de produção** (react-router-dom, XSS via open redirect, com correção disponível)
- `supabase db lint` sem erros de schema · **DAST (OWASP ZAP) pendente**, depende da URL de produção

VISUAL — este é o slide-chave. Mostre o antes/depois da política em bloco de código: vermelho para `USING (auth.uid() IS NOT NULL)`, verde para `USING (auth.uid() = id)`. Uma imagem vale mais que os bullets aqui.

FALA

"Este é o coração da defesa. A avaliação de segurança encontrou **dois defeitos reais de autorização** — e os dois foram corrigidos e re-medidos."

"O primeiro: a política de leitura da tabela de perfis dizia `USING auth.uid() IS NOT NULL`. Traduzindo: 'qualquer usuário autenticado pode ler'. Na prática, qualquer pessoa com uma conta na plataforma conseguia ler e-mail e telefone de **todos** os usuários. Isso nunca apareceu como bug, porque a interface simplesmente não mostrava esses campos — mas a API devolvia."

"O segundo: a inserção de avaliações não validava que o prestador informado era o dono do serviço. Curiosamente, bookings já tinham esse trigger; reviews não. Era uma assimetria — o tipo de defeito que a revisão de código não pega porque cada política, isolada, parece correta."

"Depois das correções, a suíte de segurança passa integralmente: **zero acessos indevidos** nos cenários testados, em 30 testes de integração distribuídos por nove arquivos, cobrindo isolamento de perfis, de serviços e de bookings, bloqueio de escalonamento e as regras de review."

"Nas dependências, o npm audit reportou 12 vulnerabilidades altas e 8 moderadas, nenhuma crítica. Dessas, **apenas uma é de produção** — no react-router-dom, com correção disponível. As demais são ferramentas de build, sem superfície de ataque em produção. E registro a única medição pendente do trabalho: a varredura dinâmica com OWASP ZAP, que depende da publicação da URL de produção."

FONTE NO ARTIGO · §7.3.2 Segurança; §4.7 (estado posterior à correção); §9 Conclusão

NA TELA

- **Confiabilidade — atende**
 - máquina de estados do booking: **0 transições inválidas aceitas**
 - integridade referencial em cascata: **0 registros órfãos**
 - fluxo crítico ponta a ponta (autenticação → serviço → booking → review): sucesso
- **Testabilidade — evoluiu**
 - suíte ampliada de **11 para 44** testes unitários e de componente, mais 30 de integração
 - cobertura de linhas: **18,0% → 32,0%** no agregado
 - módulos críticos: **82% a 100%** (auth, formulários, acesso a dados, schemas)
 - limiar antes em aberto foi fechado: piso de **30% global e 75% nos críticos**, verificado automaticamente
 - factory de mock compartilhada reduziu o esforço de escrever novos testes

FALA

"Na confiabilidade, os três cenários atendem: a máquina de estados rejeita toda transição inválida, a exclusão em cascata não deixa órfãos e o fluxo crítico completo funciona no caso válido."

"Na testabilidade, o número que importa não é a cobertura agregada — é a distribuição. A suíte foi de 11 para 44 testes de unidade e componente, mais 30 de integração, e a cobertura subiu de 18 para 32 por cento. Trinta e dois por cento parece pouco, e é pouco se olhado como média. Mas essa cobertura está **concentrada nos módulos críticos**, que ficaram entre 82 e 100 por cento: contexto de autenticação, formulários, camada de acesso a dados e schemas de validação."

"Dois pontos metodológicos aqui. Primeiro: o limiar de cobertura estava em aberto no planejamento e foi fechado na execução, em 30 por cento global e 75 nos críticos, com os módulos enumerados nominalmente e verificados de forma automática — é um piso anti-regressão, não um alvo. Segundo: introduzimos uma factory de mock compartilhada, que reduziu o custo de escrever cada novo teste. Isso é evidência direta de que a testabilidade, que é o atributo sob avaliação, de fato melhorou."

NA TELA

- **Backend sob carga — atende**
p95 de **253 ms** (critério 800 ms) · **0% de erro** em **44.413 requisições** · **~2.221 req/s** · 30 conexões concorrentes por 20 s
- **Frontend, carregamento inicial — não atende**
diagnóstico: bundle único de **679 KB**, rotas carregadas de forma *eager*
ação: **code-splitting por rota** → bundle inicial **489 KB** (−28%)
score **85** → **88** (meta 90) · LCP **3,49 s** → **3,17 s** (meta 2,5 s) · TBT 11 ms → **0 ms** · CLS **0,000**
- **Manutenibilidade — estrutura sólida, higiene abaixo do ideal**
0 dependências circulares, confirmado por duas ferramentas independentes (Madge/85 arquivos, dependency-cruiser/93 módulos)
lint não zerado: **19 violações** (25 sob configuração recomendada), incluindo complexidade ciclomática elevada
duplicação de **4,55%** na interface (meta 3%) — causa localizada: **61 linhas** quase idênticas entre os painéis de cliente e de prestador
- **O gargalo de desempenho está no frontend, não no backend**

VISUAL — duas barras comparativas resolvem: p95 253 ms contra a linha de 800 ms (folgado), e LCP 3,17 s contra a linha de 2,5 s (estourado). O contraste visual entrega o argumento sem você precisar dizer.

FALA

"Em desempenho, o resultado só faz sentido se separado por camada — e é aí que está a lição."

"O backend atende com folga: sob 30 conexões concorrentes por 20 segundos, a listagem de serviços respondeu com p95 de **253 milissegundos** contra um critério de 800, com **zero por cento de erro** em mais de 44 mil requisições, a cerca de 2.200 requisições por segundo."

"O carregamento inicial do frontend **não atende**. O Lighthouse, na mediana de três execuções em perfil móvel, deu score 85 e LCP de 3,49 segundos. Diagnosticamos um bundle único de 679 KB com todas as rotas carregadas de uma vez, aplicamos code-splitting por rota e reduzimos o bundle inicial em 28 por cento. Melhorou — score 88 e LCP 3,17 — mas **continua abaixo da meta de 2,5 segundos**. Estamos reportando isso como reprovação, não como progresso."

"E esse contraste é o achado: a API é rápida, o carregamento é lento. Isso **localiza** o gargalo no peso do bundle e confirma quantitativamente o ponto de compromisso que o ATAM tinha identificado de forma qualitativa — o SPA transfere ao cliente parte do desempenho percebido."

"Na manutenibilidade, o mesmo padrão de veredito dividido: a estrutura é sólida, com zero dependências circulares confirmadas por duas ferramentas independentes. Mas a higiene de código está abaixo do ideal — 19 violações de lint e 4,55 por cento de duplicação contra uma meta de 3. E a causa é localizada: 61 linhas quase idênticas entre os painéis de cliente e de prestador. Sabemos exatamente onde está e o que fazer."

NA TELA

- **Segurança** — atende após correção de dois defeitos reais; 1 CVE de produção a tratar; DAST pendente
- **Confiabilidade** — atende (máquina de estados, integridade referencial, fluxo crítico)
- **Testabilidade** — evoluiu (18% → 32%; críticos entre 82% e 100%, com limiar protegido)
- **Desempenho** — backend atende (p95 253 ms); **frontend não atende** (LCP 3,17 s)
- **Manutenibilidade** — estrutura atende (0 ciclos); higiene abaixo do ideal
- **Resposta à questão:** um procedimento reprodutível — modelagem fiel + cenários derivados dos atributos + medição instrumentada com evidência + ciclo de correção
- O resultado não é um atestado de qualidade uniforme — e é justamente por isso que ele vale

VISUAL — a Tabela 14 do artigo com semáforo por linha (verde/amarelo/vermelho). É o slide que a banca vai fotografar.

FALA

"Consolidando: a avaliação emite **vereditos por atributo e por camada**, e não um juízo único. Segurança atende depois das correções, com um CVE de produção a tratar. Confiabilidade atende. Testabilidade evoluiu. Desempenho tem veredito dividido: backend atende, frontend não. Manutenibilidade também: a estrutura atende, a higiene não."

"E a resposta à nossa questão de pesquisa é esse procedimento: modelagem fiel do sistema real, cenários derivados dos atributos de qualidade, medição instrumentada com registro de evidências, e o ciclo de detectar e corrigir."

"Faço questão de dizer isto com todas as letras: **o resultado não é um atestado de que o sistema é uniformemente bom**. Três defeitos reais foram encontrados e o desempenho de frontend não atinge a meta. É exatamente essa capacidade de localizar deficiências com precisão — dizer *qual* camada, *qual* política, *quais* 61 linhas — que evidencia o valor do método. Um procedimento que aprovasse tudo não teria demonstrado nada."

FONTE NO ARTIGO · §7.4 Síntese dos resultados (Tabela 14)

NA TELA

- **Contribuição metodológica** — roteiro reproduzível que articula ISO/IEC 25010 (taxonomia) + ATAM (análise qualitativa) + ferramental instrumentado (evidência quantitativa), transferível a outras aplicações BaaS
- **Contribuição prática** — as correções aplicadas e os gates automatizados na integração contínua
- **Os três defeitos são característicos do paradigma:** todos residem em configuração declarativa ou no serviço gerenciado, e **nenhum se manifestava como erro funcional**
- **Limitações** — DAST pendente · avaliação conduzida pela própria equipe (mitigada por fixar critérios antes da coleta) · generalização restrita a um único caso
- **Trabalhos futuros** — executar o DAST em ambiente publicado · aplicar o procedimento a outras plataformas BaaS · reduzir a duplicação entre os painéis · seguir otimizando o LCP até a meta
- **Obrigado. Perguntas?**

FALA

(Richardy) "A principal contribuição do trabalho é metodológica: um roteiro reproduzível, com registro de evidências e critérios fixados antes da coleta, transferível a outras aplicações que adotem o mesmo paradigma. Ele articula três elementos — a 25010 dá a taxonomia, o ATAM organiza a análise qualitativa, e o ferramental produz a evidência."

"E reforço o achado que amarra tudo: os três defeitos que encontramos são característicos do paradigma BaaS. Todos moram em configuração declarativa ou no serviço gerenciado, e nenhum deles se manifestava como erro funcional na aplicação. Eram invisíveis para o teste convencional."

(Pedro, ou Richardy) "Como limitações, registramos a varredura dinâmica ainda pendente, o fato de a avaliação ter sido conduzida pela própria equipe — o que nos obrigou a explicitar critérios previamente para mitigar viés — e a generalização restrita a um único caso. Como trabalhos futuros, executar o DAST em ambiente publicado e aplicar o mesmo procedimento a outras plataformas, para testar sua transferibilidade."

"Obrigado. Estamos à disposição para as perguntas."

Slides reserva — para o Q&A da banca

Não apresentados. Deixe-os depois do slide 22 no arquivo, para chamar por número se a pergunta vier.

R1

Tabela 14 completa — síntese por atributo

reserva

ATRIBUTO (ISO/IEC 25010)	RESULTADO
Segurança	Autorização declarativa (RLS e triggers) atende após a correção de dois defeitos reais; 1 CVE de produção a tratar; DAST pendente contra produção
Confiabilidade	Atende (máquina de estados, integridade referencial e fluxo crítico)
Testabilidade	Evoluiu (cobertura de 18% para 32%; módulos críticos entre 82% e 100%, com limiar protegido)
Eficiência de desempenho	Backend atende (p95 de 253 ms); frontend não atende (LCP de 3,17 s após otimização)
Manutenibilidade	Estrutura atende (0 ciclos); higiene de código abaixo do ideal (lint e duplicação nos painéis)

FONTE NO ARTIGO · §7.4, Tabela 14

R2

O defeito de RLS, antes e depois

reserva

NA TELA

- **ANTES** — `CREATE POLICY ... ON profiles FOR SELECT USING (auth.uid() IS NOT NULL)` qualquer usuário autenticado lia `email` e `phone` de todos os perfis
- **DEPOIS** — leitura direta restrita a `auth.uid() = id`; dados de contraparte servidos pela view `public_profiles`, que expõe `id`, `full_name`, `avatar_url`, `user_type`, `created_at` e **omite email e phone**
- Por que passou despercebido: a interface nunca exibia esses campos — mas a API os devolvia
- Por que o cenário original não pegaria: ele testava apenas o acesso **anônimo**; foi ampliado para o autenticado não relacionado (registrado em §5.2.1)

FONTE NO ARTIGO · §7.3.2; §4.4 (view `public_profiles`); §4.7; nota de refinamento em §5.2.1

R3

Números brutos de desempenho

reserva

NA TELA

- **Carga (autocannon)** — 30 conexões concorrentes · 20 s · **44.413 requisições** · **0% de erro** · **~2.221 req/s** · **p95 = 253 ms** (critério 800 ms)
- Calibração do limiar: execução exploratória de 5 conexões por 5 s deu p95 ≈ **57 ms**; somada à heurística de 1 s para percepção de fluidez, fixou-se 800 ms com margem conservadora
- **Lighthouse** — mediana de 3 execuções, perfil móvel com limitação de CPU e rede
- Antes do code-splitting: score **85** · LCP **3,49 s** · TBT **11 ms** · CLS **0,000** · bundle **679 KB**
- Depois: score **88** · LCP **3,17 s** · TBT **0 ms** · CLS **0,000** · bundle **489 KB** (–28%)
- Metas: score ≥ 90 · LCP ≤ 2,5 s → **não atingidas**

FONTE NO ARTIGO · §7.3.5; §5.2.2 nota de calibração do limiar; evidências em `docs/tcc/medicoes/evidencias/`

NA TELA

- **Validade externa** — variabilidade de rede e do plano do serviço gerenciado
- **Validade de conclusão** — dependência da configuração do ambiente de carga
- **Validade de construção** — representatividade dos cenários frente ao uso real
- **Mitigações** — repetição das medições (mediana de 3 no Lighthouse) · fixação de parâmetros · registro das condições de cada execução
- **Sobre a auto-avaliação:** critérios fixados **antes** da coleta · regra anti-fabricação (nenhum valor sem evidência arquivada) · resultados desfavoráveis reportados com o mesmo rigor — três reprovações estão no artigo

FALA

Se a banca questionar a isenção: "A melhor evidência de que não acomodamos os critérios ao resultado é que o trabalho **reprova o próprio sistema em três frentes**: segurança na primeira medição, desempenho de frontend mesmo após otimização, e higiene de código. Se o viés estivesse operando, esses três não estariam no artigo."

FONTE NO ARTIGO · §5.4 Ameaças à validade; §3.5 Procedimentos de análise; §9 Limitações

R5 Referências principais

NA TELA

- **BASS, L.; CLEMENTS, P.; KAZMAN, R.** *Software architecture in practice*. 3. ed. Addison-Wesley, 2012.
- **CLEMENTS, P.; KAZMAN, R.; KLEIN, M.** *Evaluating software architectures: methods and case studies*. Addison-Wesley, 2002.
- **ISO/IEC 25010** — *Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. ISO, 2011.
- **PRESSMAN, R. S.; MAXIM, B. R.** *Engenharia de software: uma abordagem profissional*. 8. ed. AMGH, 2016.
- **SOMMERVILLE, I.** *Engenharia de software*. 9. ed. Pearson Prentice Hall, 2011.
- ABNT NBR 6023:2018 · ABNT NBR 10520:2023

FONTE NO ARTIGO · Referências

Checklist de montagem e ensaio

Ao montar os slides

- Os bullets do bloco **NA TELA** já estão no tamanho certo — resista a colar a fala no slide.
- Números sempre em destaque tipográfico: **253 ms**, **3,17 s**, **0 acessos indevidos**, **4,55%**. É o que a banca anota.
- Slides que pedem imagem, não texto: **3** (tradicional × BaaS), **11** (Figura 1), **15** (Figura 3), **18** (antes/depois da política), **20** (duas barras), **21** (Tabela 14 com semáforo).
- Use verde/vermelho de forma consistente para *atende* / *não atende* — o slide 21 depende disso.

Ao ensaiar

- Cronometre os slides **6**, **18** e **21** isolados: são os que sustentam a defesa e os que mais tendem a estourar.
- Ensaie o *handoff* depois do slide 11 — uma frase, sem apresentação do colega.
- Se atrasar: corte detalhe dos slides **7**, **8** e **12**. Nunca corte o **6**, o **18** nem o **21**.
- Perguntas mais prováveis: "*por que vocês mesmos desenvolveram o sistema?*" (slide 6) · "*32% de cobertura não é baixo?*" (slide 19, argumento da distribuição) · "*por que o DAST não foi feito?*" (slide 18) · "*como garantir isenção?*" (reserva R4).

Conteúdo extraído de `docs/Artigo_PedroConrado_RichardyRodrigues_ATUALIZADO.docx`. Todos os valores quantitativos conferem com a Seção 7 do artigo e com o registro em `docs/tcc/medicoes/`. Fonte deste PDF: `docs/tcc/conteudo-slides-defesa.html`.